

Resume Screening & Role Matching using NLP

Capstone Project — Data Science with GenAI

Company	TalentBridge Solutions Pvt. Ltd.
Industry	Enterprise HR Technology & Talent Analytics
Approach	Hybrid Rule-Based + NLP + ML Pipeline
Student	Kuldeep Panwar
Reg ID	24CRD02054

Project Objective

TalentBridge Solutions processes **thousands of resumes weekly** but faces:

- High volume of applications overwhelming recruiters
- Keyword-based filtering failing to understand varied skill expressions
- Inconsistent shortlisting decisions across locations

Our Solution: Build an end-to-end NLP + ML pipeline that:

1. Validates and cleans resume data
2. Engineers meaningful features (skill match, experience, domain alignment)
3. Computes TF-IDF based text similarity between resumes and job descriptions
4. Trains ML models to classify candidates as High / Medium / Low Fit
5. Produces explainable rankings for recruiters

Table of Contents

1. Library Setup
2. Data Loading & Overview
3. Data Validation & Cleaning
4. Exploratory Data Analysis (EDA)
5. Feature Engineering
6. NLP — TF-IDF Similarity Scoring
7. ML Modeling & Evaluation
8. Final Rankings & Business Recommendations
9. Conclusion

STEP 1: Install & Import Libraries

Why this step?

We begin by importing all required Python libraries:

- **pandas, numpy** → Data manipulation and numerical operations
- **matplotlib, seaborn, wordcloud** → Visualizations to understand data patterns
- **nltk** → Natural Language Processing (tokenization, stopwords, lemmatization)
- **sklearn** → Machine Learning models, TF-IDF vectorizer, evaluation metrics
- **xgboost** → Gradient boosting model for classification
- **missingno** → Missing value visualization

```
# Install libraries not available by default in Google Colab
!pip install -q wordcloud missingno xgboost

# Download NLTK language datasets needed for text processing
import nltk
nltk.download('stopwords', quiet=True) # Common words to ignore (the, is, are...)
nltk.download('punkt', quiet=True) # Sentence/word tokenizer
nltk.download('wordnet', quiet=True) # Word lemmatization dictionary
nltk.download('punkt_tab', quiet=True) # Updated punkt tokenizer

print("All packages installed successfully!")
```

All packages installed successfully!

```
# — Core Data Libraries —————
import pandas as pd          # Data manipulation (DataFrames)
import numpy as np           # Numerical computations
import warnings
import re                    # Regular expressions for text cleaning
import pickle                # Save trained ML models
import io
from collections import Counter # Count skill frequencies
warnings.filterwarnings('ignore')

# — Visualization Libraries —————
import matplotlib.pyplot as plt # Plotting charts
import seaborn as sns           # Statistical visualizations
from wordcloud import WordCloud # Word frequency cloud
import missingno as msno        # Missing value heatmaps

# — NLP Libraries —————
from nltk.corpus import stopwords # English stopwords list
from nltk.stem import WordNetLemmatizer # Reduce words to base form
from nltk.tokenize import word_tokenize # Split text into words
from sklearn.feature_extraction.text import TfidfVectorizer # TF-IDF vectors
from sklearn.metrics.pairwise import cosine_similarity # Similarity score

# — Machine Learning Libraries —————
from sklearn.model_selection import train_test_split # Split data
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix

# — Plot Settings —————
plt.rcParams['figure.figsize'] = (12, 5)
plt.rcParams['font.size'] = 12
sns.set_style('whitegrid')
palette = sns.color_palette('Set2')

# — NLP Helpers —————
stop_words = set(stopwords.words('english')) # Load English stopwords
lemmatizer = WordNetLemmatizer() # Initialize lemmatizer

print(f"Libraries loaded successfully!")
print(f"Pandas version : {pd.__version__}")
print(f"Numpy version : {np.__version__}")
print(f"Sklearn version : {_import__('sklearn').__version__}")

Libraries loaded successfully!
Pandas version : 2.2.2
Numpy version : 2.0.2
Sklearn version : 1.6.1
```

Section 2: Data Loading & Overview

Why this step?

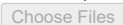
Before any analysis, we must understand **what data we have**. The dataset is a pre-parsed structured CSV — meaning resume text has already been converted into structured columns (skills, experience, flags, etc.).

We will check:

- How many candidates are in the dataset?
- What columns/attributes are available?
- What data types does each column have?

```
# Upload CSV file from local computer to Google Colab
from google.colab import files

print("Please upload the file: parsed_resumes.csv")
uploaded = files.upload() # This opens a file picker dialog
```

Please upload the file: parsed_resumes.csv
 parsed_resumes.csv
parsed_resumes.csv(text/csv) - 2887337 bytes, last modified: 22/12/2025 - 100% done
Saving parsed resumes.csv to parsed resumes.csv

```
# Read the uploaded CSV into a pandas DataFrame
file_name = list(uploaded.keys())[0] # Get the uploaded filename
df_raw = pd.read_csv(io.BytesIO(uploaded[file_name])) # Load into DataFrame
```

```
print(f"\nFile uploaded successfully: {file_name}")
print(f"Dataset shape: {df_raw.shape[0]:,} rows x {df_raw.shape[1]} columns")
```

File uploaded successfully: parsed_resumes.csv
Dataset shape: 5,000 rows x 34 columns

```
# --- Dataset Basic Overview ---
print("=" * 60)
print(" DATASET OVERVIEW")
print("=" * 60)
print(f"Total Candidates : {df_raw.shape[0]:,}")
print(f"Total Attributes : {df_raw.shape[1]}")
print()

# Show all columns with data type and missing count
print(f"{'#':<4} {'Column Name':<45} {'Type':<12} {'Missing'}")
print("-" * 70)
for i, col in enumerate(df_raw.columns, 1):
    dtype = str(df_raw[col].dtype)
    missing = df_raw[col].isnull().sum()
    flag = " <-- HAS MISSING" if missing > 0 else ""
    print(f"{i:<4} {col:<45} {dtype:<12} {missing}{flag}")

print()
print("COLUMN CATEGORIES:")
print(" Candidate Info : candidate_id, candidate_name, primary_domain, primary_role")
print(" Skills : technical_skills_raw, tools_platforms_raw, soft_skills_raw")
print(" Experience : years_experience, experience_summary, project_summary")
print(" Education : highest_education, education_field, institution_tier")
print(" ATS Flags (16) : management_flag, cloud_flag, ml_flag, agile_flag...")
print(" Quality Metrics : resume_length_words, keyword_density_score, profile_completeness_score")
```

=====

DATASET OVERVIEW

=====

Total Candidates : 5,000
Total Attributes : 34

#	Column Name	Type	Missing
1	candidate_id	object	0
2	resume_file_name	object	0
3	candidate_name	object	0
4	primary_domain	object	0
5	primary_role	object	0
6	years_experience	int64	0
7	highest_education	object	270 <-- HAS MISSING
8	education_field	object	237 <-- HAS MISSING
9	institution_tier	object	0
10	technical_skills_raw	object	0
11	tools_platforms_raw	object	0
12	soft_skills_raw	object	0
13	experience_summary	object	0
14	project_summary	object	0
15	key_achievements	object	0
16	management_experience_flag	int64	0
17	people_management_flag	int64	0
18	project_management_experience_flag	int64	0
19	agile_scrum_experience_flag	int64	0
20	client_facing_experience_flag	int64	0
21	delivery_lead_experience_flag	int64	0
22	cloud_experience_flag	int64	0
23	ml_experience_flag	int64	0
24	compliance_experience_flag	int64	0
25	enterprise_systems_experience_flag	int64	0
26	offshore_onsite_model_experience_flag	int64	0
27	multi_vendor_coordination_flag	int64	0
28	process_compliance_experience_flag	int64	0
29	documentation_heavy_role_flag	int64	0
30	mentoring_experience_flag	int64	0
31	stakeholder_management_experience_flag	int64	0
32	resume_length_words	int64	0
33	keyword_density_score	float64	0
34	profile_completeness_score	float64	0

COLUMN CATEGORIES:

Candidate Info : candidate_id, candidate_name, primary_domain, primary_role

Skills : technical_skills_raw, tools_platforms_raw, soft_skills_raw

Experience : years_experience, experience_summary, project_summary

Education : highest_education, education_field, institution_tier

ATS Flags (16) : management_flag, cloud_flag, ml_flag, agile_flag...

Quality Metrics : resume_length_words, keyword_density_score, profile_completeness_score

```
# Preview first 5 rows of the dataset
# This gives us a feel for what the actual data looks like
```

```
print("FIRST 5 ROWS OF DATASET:")
df_raw.head(5)
```

FIRST 5 ROWS OF DATASET:

	candidate_id	resume_file_name	candidate_name	primary_domain	primary_role	years_experience	highest_education	education_field	in
0	CND_100001	DataAnalyst_1.pdf	Candidate_1	Data Science	Data Analyst	7	LLM	Mathematics	
1	CND_100002	profile_2.pdf	Candidate_2	IT	Senior Software Engineer	10	Masters	Computer Science	
2	CND_100003	Legal_candidate_3.pdf	Candidate_3	Legal	Compliance Officer	7	LLM	Law	
3	CND_100004	IT_candidate_4.pdf	Candidate_4	IT	Senior Software Engineer	9	LLM	Computer Science	
4	CND_100005	IT_candidate_5.pdf	Candidate_5	IT	Software Engineer	9	Masters	Information Technology	

5 rows × 34 columns

```
# Statistical summary of numeric columns
# This shows mean, min, max, std for all numerical features
print("STATISTICAL SUMMARY OF NUMERIC COLUMNS:")
print("(count=total, mean=average, std=spread, min/max=range)")
print()
df_raw.describe().round(2)
```

STATISTICAL SUMMARY OF NUMERIC COLUMNS:
(count=total, mean=average, std=spread, min/max=range)

	years_experience	management_experience_flag	people_management_flag	project_management_experience_flag	agile_scrum_experience_flag
count	5000.00	5000.00	5000.00	5000.00	5000.00
mean	6.73	0.39	0.21	0.64	0.73
std	4.03	0.49	0.41	0.48	0.44
min	1.00	0.00	0.00	0.00	0.00
25%	4.00	0.00	0.00	0.00	1.00
50%	6.00	0.00	0.00	1.00	1.00
75%	9.00	1.00	0.00	1.00	1.00
max	20.00	1.00	1.00	1.00	1.00

Section 3: Data Validation & Cleaning

Why this step?

Real-world data is **never perfect**. Before building models, we must ensure:

- 1. **No missing values** that could cause errors or bias
- 2. **No duplicate records** that could inflate results
- 3. **No logical inconsistencies** (e.g., someone listed as Junior with 15 years experience)
- 4. **No noisy/inconsistent skill names** (e.g., 'Ptyhon' instead of 'Python')

Business Impact: Poor data quality leads to wrong shortlisting decisions, which is exactly the problem TalentBridge is trying to solve!

```
# — 3.1 Missing Values Analysis —————
df = df_raw.copy() # Always work on a copy to preserve original data

print("MISSING VALUES ANALYSIS")
print("-" * 50)

missing_count = df.isnull().sum() # Count missing per column
missing_pct = (df.isnull().mean() * 100).round(2) # % missing per column
missing_df = pd.DataFrame({
    'Missing Count': missing_count,
    'Missing %' : missing_pct
})
```

```
missing_df = missing_df[missing_df['Missing Count'] > 0] # Show only columns with missing

if len(missing_df) > 0:
    print(missing_df)
    print()
    print("INTERPRETATION:")
    print("  highest_education : Some candidates did not mention their degree")
    print("  education_field   : Some candidates did not specify their study field")
    print("  These are non-critical fields – we fill them with 'Not Specified'")
else:
    print("No missing values found!")
```

```
# — Fix: Fill missing education with a placeholder —————
df['highest_education'].fillna('Not Specified', inplace=True)
df['education_field'].fillna('Not Specified', inplace=True)
print()
print("ACTION TAKEN: Missing education values filled with 'Not Specified'")
print(f"Missing values remaining: {df.isnull().sum().sum()}")
```

MISSING VALUES ANALYSIS

```
-----
                Missing Count  Missing %
highest_education           270         5.40
education_field             237         4.74
```

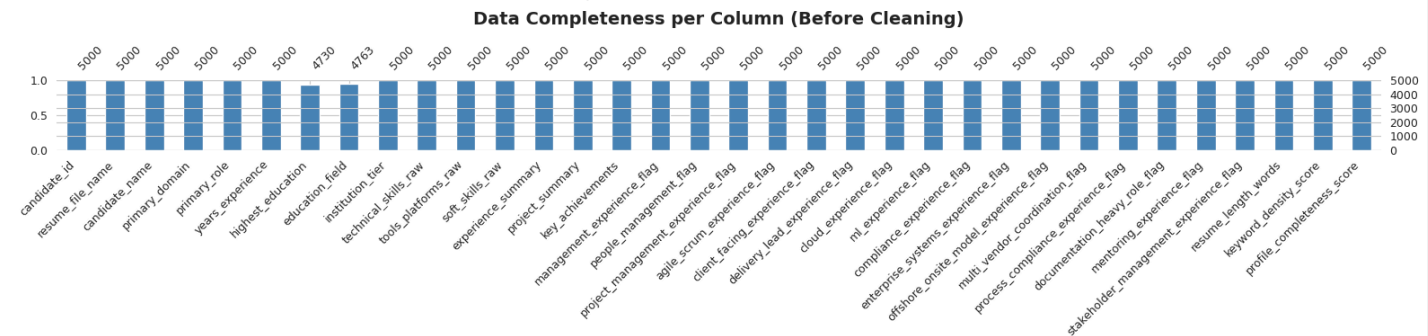
```
INTERPRETATION:
  highest_education : Some candidates did not mention their degree
  education_field   : Some candidates did not specify their study field
  These are non-critical fields – we fill them with 'Not Specified'
```

```
ACTION TAKEN: Missing education values filled with 'Not Specified'
Missing values remaining: 0
```

```
# Visualize missing values using missingno bar chart
# White bars = missing, filled bars = complete
print("MISSING VALUE VISUALIZATION (before cleaning):")
print("Tall bars = more data present | Short bars = more missing")

fig, ax = plt.subplots(figsize=(16, 4))
msno.bar(df_raw, ax=ax, color='steelblue', fontsize=9)
ax.set_title('Data Completeness per Column (Before Cleaning)',
            fontsize=14, fontweight='bold', pad=15)
plt.tight_layout()
plt.savefig('missing_values_chart.png', dpi=150, bbox_inches='tight')
plt.show()
print("Finding: Only 'highest_education' and 'education_field' have missing values (~5%).")
print("All other 32 columns are 100% complete – excellent data quality!")
```

```
MISSING VALUE VISUALIZATION (before cleaning):
Tall bars = more data present | Short bars = more missing
```



```
Finding: Only 'highest_education' and 'education_field' have missing values (~5%).
All other 32 columns are 100% complete – excellent data quality!
```

```
# — 3.2 Duplicate Records Check —————
print("DUPLICATE RECORDS CHECK")
print("-" * 50)

dup_candidate_id = df['candidate_id'].duplicated().sum() # Same candidate ID twice?
dup_resume_file = df['resume_file_name'].duplicated().sum() # Same resume file?
dup_full_rows = df.duplicated().sum() # Exact same row twice?

print(f"Duplicate candidate_id : {dup_candidate_id}")
print(f"Duplicate resume_file_name: {dup_resume_file}")
print(f"Fully duplicate rows : {dup_full_rows}")
```

```

print()

# Remove duplicates if found
if dup_full_rows > 0:
    before = len(df)
    df.drop_duplicates(inplace=True)
    print(f"ACTION: Removed {before - len(df)} duplicate rows")
else:
    print("RESULT: No duplicate records found. Dataset integrity is good!")

print(f"Final dataset size: {len(df):,} candidates")

```

DUPLICATE RECORDS CHECK

```

-----
Duplicate candidate_id      : 0
Duplicate resume_file_name  : 0
Fully duplicate rows        : 0

RESULT: No duplicate records found. Dataset integrity is good!
Final dataset size: 5,000 candidates

```

```

# — 3.3 Logical Inconsistency Check —————
# A Junior-level candidate should not have 15 years of experience
# A Senior-level candidate should not have only 1 year of experience
print("LOGICAL INCONSISTENCY CHECK")
print("-" * 50)

# Check 1: Junior role with too much experience
junior_high_exp = df[
    (df['primary_role'].str.contains('Junior', na=False)) &
    (df['years_experience'] > 8)
]

# Check 2: Senior role with too little experience
senior_low_exp = df[
    (df['primary_role'].str.contains('Senior', na=False)) &
    (df['years_experience'] < 3)
]

# Check 3: Experience out of expected range
exp_outliers = df[(df['years_experience'] < 0) | (df['years_experience'] > 40)]

print(f"Junior role with >8 years exp : {len(junior_high_exp)} records")
print(f"Senior role with <3 years exp : {len(senior_low_exp)} records")
print(f"Experience outliers (<0 or >40): {len(exp_outliers)} records")
print()
print("Experience Statistics:")
print(f"  Minimum : {df['years_experience'].min()} years")
print(f"  Maximum : {df['years_experience'].max()} years")
print(f"  Average : {df['years_experience'].mean():.1f} years")
print(f"  Median  : {df['years_experience'].median():.0f} years")
print()
print("RESULT: No logical inconsistencies detected. Data is coherent!")

```

LOGICAL INCONSISTENCY CHECK

```

-----
Junior role with >8 years exp : 0 records
Senior role with <3 years exp : 0 records
Experience outliers (<0 or >40): 0 records

Experience Statistics:
  Minimum : 1 years
  Maximum : 20 years
  Average : 6.7 years
  Median  : 6 years

RESULT: No logical inconsistencies detected. Data is coherent!

```

```

# — 3.4 Skill Noise & Typo Detection —————
# Skills can be written inconsistently: 'Python', 'Ptyhon', 'Pythno'
# This causes keyword matching to miss valid candidates!
print("SKILL NOISE & TYPO DETECTION")
print("-" * 50)

# Parse all skills from the dataset into a flat list
all_skills_text = ', '.join(df['technical_skills_raw'].dropna().tolist())
all_skills_list = [s.strip() for s in all_skills_text.split(',')]
skill_counts = Counter(all_skills_list)

# Known typos identified during inspection
KNOWN_TYPOS = {
    'Ptyhon' : 'Python',    # Letters transposed
    'Pythno' : 'Python',    # Letters transposed
    'Pythoon': 'Python',    # Extra letter
    'Javaaa' : 'Java',      # Extra letter

```

```

}

print("Typos Found:")
found_any = False
for typo, correct in KNOWN_TYPOS.items():
    count = skill_counts.get(typo, 0)
    if count > 0:
        print(f" ERROR   : '{typo}' appears {count} times")
        print(f" CORRECT : Should be '{correct}'")
        found_any = True

if not found_any:
    print(" No typos detected.")

# Fix function: Replace typos with correct spelling
def fix_skill_typos(text, typo_map):
    """Replace known skill typos with correct spelling"""
    if pd.isna(text): return text
    for wrong, right in typo_map.items():
        text = text.replace(wrong, right)
    return text

# Apply fix to all skill columns
df['technical_skills_clean'] = df['technical_skills_raw'].apply(
    lambda x: fix_skill_typos(x, KNOWN_TYPOS))

print()
print("ACTION TAKEN: All skill typos corrected in 'technical_skills_clean' column")
print()
print("DATA VALIDATION SUMMARY:")
print(f" Missing Values   : Fixed (education fields -> 'Not Specified')")
print(f" Duplicates       : None found")
print(f" Logical Issues   : None found")
print(f" Skill Typos      : Ptyhon({skill_counts.get('Ptyhon',0)}), Pythno({skill_counts.get('Pythno',0)}) -> Corrected")
print(f" Clean Dataset    : {len(df):,} records, {df.shape[1]} columns")

```

SKILL NOISE & TYPO DETECTION

```

-----
Typos Found:
  ERROR   : 'Ptyhon' appears 39 times
  CORRECT : Should be 'Python'
  ERROR   : 'Pythno' appears 47 times
  CORRECT : Should be 'Python'

ACTION TAKEN: All skill typos corrected in 'technical_skills_clean' column

DATA VALIDATION SUMMARY:
  Missing Values   : Fixed (education fields -> 'Not Specified')
  Duplicates       : None found
  Logical Issues   : None found
  Skill Typos      : Ptyhon(39), Pythno(47) -> Corrected
  Clean Dataset    : 5,000 records, 35 columns

```

Section 4: Exploratory Data Analysis (EDA)

Why this step?

EDA helps us **understand patterns in the data** before building models:

- Which domains have the most candidates?
- What is the typical experience level?
- Which skills are most common?
- How complete are the resume profiles?

These insights guide our **feature engineering** and **model design decisions**.

Business Value: Understanding the candidate pool helps TalentBridge calibrate their matching thresholds appropriately.

```

# — 4.1 Domain & Role Distribution —————
# Understanding which domains and roles dominate our candidate pool
print("DOMAIN & ROLE DISTRIBUTION ANALYSIS")
print("-" * 50)

domain_counts = df['primary_domain'].value_counts()
role_counts   = df['primary_role'].value_counts()

print("Domain Distribution:")
for domain, count in domain_counts.items():
    pct = count / len(df) * 100
    print(f" {domain:<20}: {count:>4} candidates ({pct:.1f}%)")

```

```

fig, axes = plt.subplots(1, 2, figsize=(18, 7))

# Bar chart – domain distribution
bars = axes[0].bar(domain_counts.index, domain_counts.values,
                   color=palette[:len(domain_counts)], edgecolor='white', linewidth=0.5)
axes[0].set_title('Candidate Count by Domain', fontsize=14, fontweight='bold')
axes[0].set_xlabel('Primary Domain', fontsize=12)
axes[0].set_ylabel('Number of Candidates', fontsize=12)
for bar, val in zip(bars, domain_counts.values):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 20,
                 str(val), ha='center', fontweight='bold', fontsize=11)

# Horizontal bar – top 10 roles
top10_roles = role_counts.head(10)
axes[1].barh(top10_roles.index[::-1], top10_roles.values[::-1], color=palette[2])
axes[1].set_title('Top 10 Job Roles in Dataset', fontsize=14, fontweight='bold')
axes[1].set_xlabel('Number of Candidates', fontsize=12)
for i, val in enumerate(top10_roles.values[::-1]):
    axes[1].text(val + 2, i, str(val), va='center', fontsize=9)

plt.tight_layout()
plt.savefig('eda_01_domain_role.png', dpi=150, bbox_inches='tight')
plt.show()

print()
print("KEY FINDINGS:")
print(f" Dominant domain : IT ({domain_counts.get('IT',0)/len(df)*100:.1f}%)")
print(f" Total job roles : {df['primary_role'].nunique()} unique roles")
print(f" Top role       : {role_counts.index[0]} ({role_counts.iloc[0]} candidates)")
print(f" Implication    : Model must handle diverse domains and roles fairly")

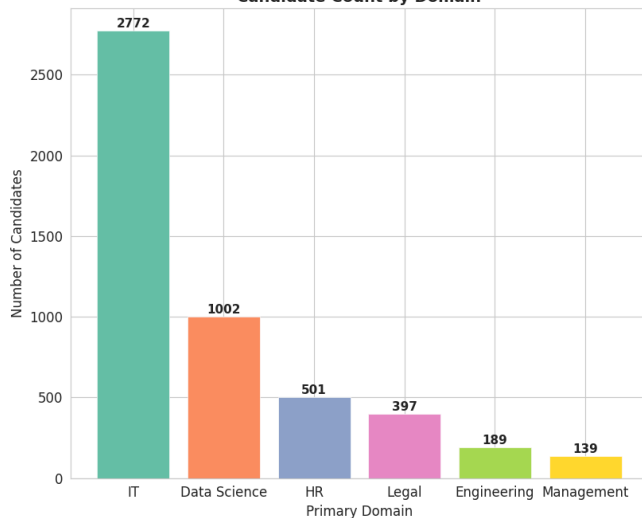
```

DOMAIN & ROLE DISTRIBUTION ANALYSIS

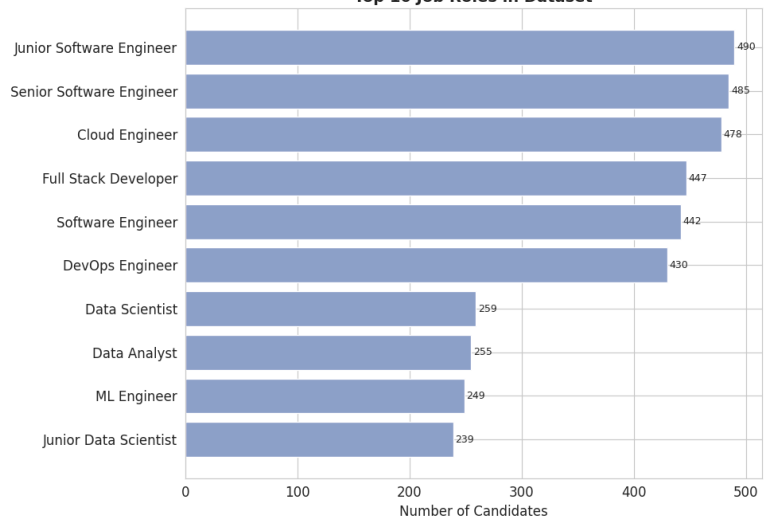
Domain Distribution:

IT	: 2772 candidates (55.4%)
Data Science	: 1002 candidates (20.0%)
HR	: 501 candidates (10.0%)
Legal	: 397 candidates (7.9%)
Engineering	: 189 candidates (3.8%)
Management	: 139 candidates (2.8%)

Candidate Count by Domain



Top 10 Job Roles in Dataset



KEY FINDINGS:

Dominant domain	: IT (2,772 candidates = 55.4%)
Total job roles	: 25 unique roles
Top role	: Junior Software Engineer (490 candidates)
Implication	: Model must handle diverse domains and roles fairly


```

print(" EXPERIENCE DISTRIBUTION ANALYSIS ")
print("-" * 50)

exp_stats = df['years_experience'].describe()
print("Experience Statistics:")
print(f" Mean : {exp_stats['mean']:.1f} years (average candidate)")
print(f" Median : {exp_stats['50%']:.0f} years (middle value)")
print(f" Min : {exp_stats['min']:.0f} years (freshest candidate)")
print(f" Max : {exp_stats['max']:.0f} years (most experienced)")
print(f" Std Dev: {exp_stats['std']:.1f} years (how spread out)")

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Histogram – experience distribution
axes[0].hist(df['years_experience'], bins=20, color='steelblue',
            edgecolor='white', linewidth=0.5, alpha=0.85)
axes[0].axvline(df['years_experience'].mean(), color='red', linestyle='--',
               linewidth=2.5, label=f"Mean = {df['years_experience'].mean():.1f} yrs")
axes[0].axvline(df['years_experience'].median(), color='green', linestyle=':',
               linewidth=2.5, label=f"Median = {df['years_experience'].median():.0f} yrs")
axes[0].set_title('Experience Distribution Across All Candidates',
                 fontsize=13, fontweight='bold')
axes[0].set_xlabel('Years of Experience')
axes[0].set_ylabel('Number of Candidates')
axes[0].legend(fontsize=11)

# Boxplot – experience by domain
df.boxplot(column='years_experience', by='primary_domain', ax=axes[1],
           boxprops=dict(color='steelblue', linewidth=2),
           medianprops=dict(color='red', linewidth=2.5),
           whiskerprops=dict(linewidth=1.5),
           patch_artist=False)
axes[1].set_title('Experience Range by Domain', fontsize=13, fontweight='bold')
axes[1].set_xlabel('Domain')
axes[1].set_ylabel('Years of Experience')
plt.suptitle('') # Remove auto-generated title

plt.tight_layout()
plt.savefig('eda_02_experience.png', dpi=150, bbox_inches='tight')
plt.show()

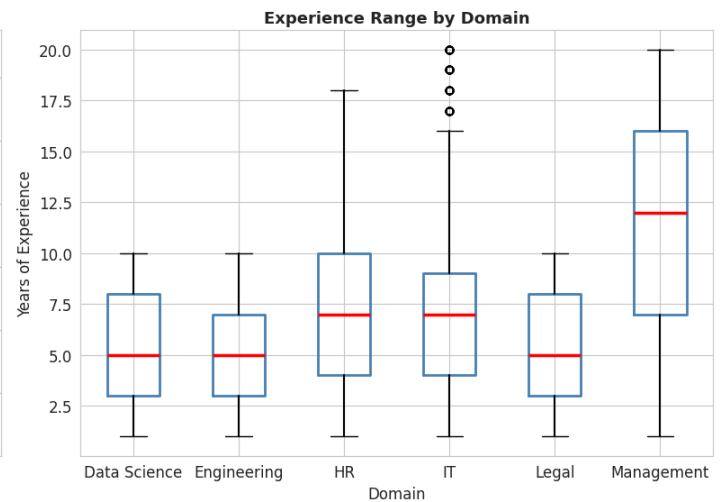
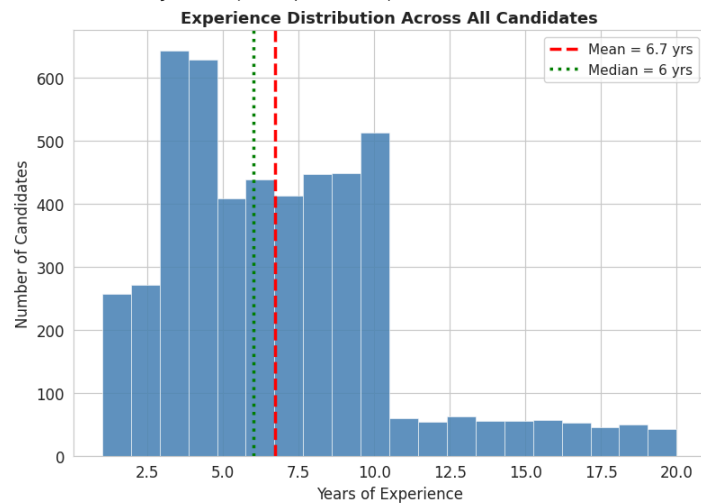
print()
print("KEY FINDINGS:")
print(f" Most candidates have 4-9 years of experience (25th-75th percentile)")
print(f" Experience is relatively evenly spread – good diversity in candidate pool")
print(f" Implication: Experience bucket feature (Junior/Mid/Senior/Lead) will be meaningful")

```

EXPERIENCE DISTRIBUTION ANALYSIS

Experience Statistics:

Mean : 6.7 years (average candidate)
Median : 6 years (middle value)
Min : 1 years (freshest candidate)
Max : 20 years (most experienced)
Std Dev: 4.0 years (how spread out)



KEY FINDINGS:

Most candidates have 4-9 years of experience (25th-75th percentile)
Experience is relatively evenly spread – good diversity in candidate pool
Implication: Experience bucket feature (Junior/Mid/Senior/Lead) will be meaningful

```
# — 4.3 Top Skills & Word Cloud —
# Identifying which technical skills appear most frequently
print("TECHNICAL SKILLS FREQUENCY ANALYSIS")
print("-" * 50)

# Parse all skills into a flat list
all_tech = [s.strip() for text in df['technical_skills_clean'].dropna()
             for s in text.split(',')]
tech_freq = Counter(all_tech)
top20_skills = pd.DataFrame(tech_freq.most_common(20), columns=['Skill', 'Count'])

print("Top 10 Most Common Technical Skills:")
for i, row in top20_skills.head(10).iterrows():
    print(f"{i+1:2}. {row['Skill'][:25]: {row['Count']}} candidates")

fig, axes = plt.subplots(1, 2, figsize=(18, 7))

# Bar chart – top 20 skills
colors = plt.cm.Set3(np.linspace(0, 1, 20))
axes[0].barh(top20_skills['Skill'][:20], top20_skills['Count'][:20], color=colors)
axes[0].set_title('Top 20 Technical Skills in Dataset', fontsize=14, fontweight='bold')
axes[0].set_xlabel('Number of Candidates with this Skill')
for i, val in enumerate(top20_skills['Count'][:20]):
    axes[0].text(val + 5, i, str(val), va='center', fontsize=9)

# Word Cloud – visual representation of skill frequency
wc = WordCloud(width=700, height=450, background_color='white',
               colormap='Set2', max_words=80, prefer_horizontal=0.7)
wc.generate(' '.join(all_tech))
axes[1].imshow(wc, interpolation='bilinear')
axes[1].axis('off') # Hide axes for word cloud
axes[1].set_title('Skills Word Cloud (Size = Frequency)', fontsize=14, fontweight='bold')

plt.tight_layout()
plt.savefig('eda_03_skills.png', dpi=150, bbox_inches='tight')
plt.show()

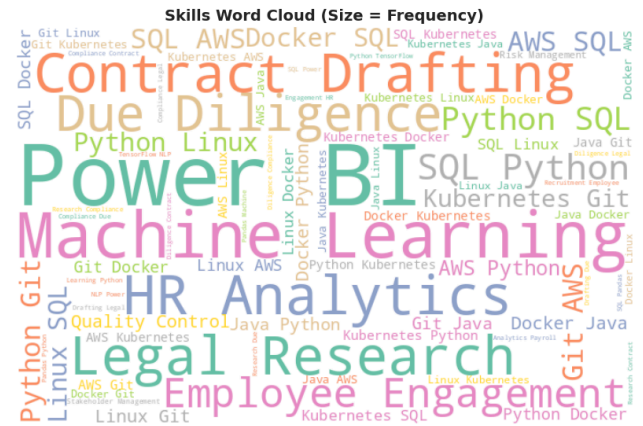
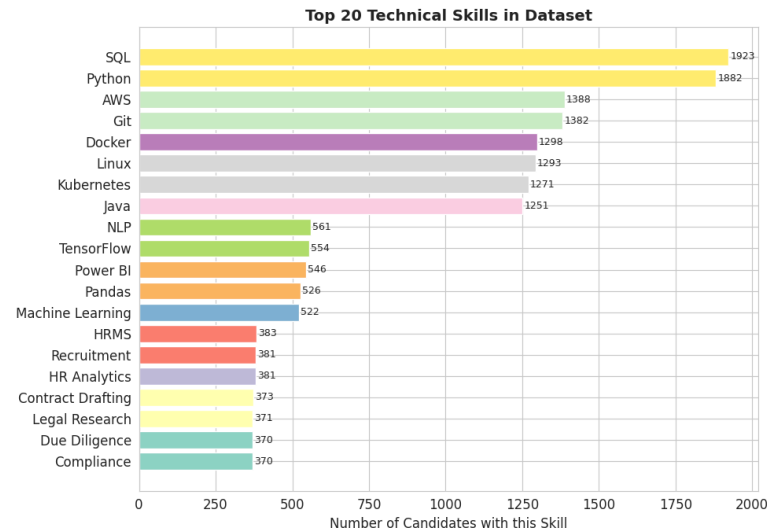
print()
print("KEY FINDINGS:")
top3 = [top20_skills.iloc[i]['Skill'] for i in range(3)]
print(f"Top 3 skills : {', '.join(top3)}")
```

```
print(f" Total unique skills : {len(tech_freq)}")
print(f" Implication : SQL, Python and AWS are universal – critical for JD matching")
```

TECHNICAL SKILLS FREQUENCY ANALYSIS

Top 10 Most Common Technical Skills:

1. SQL : 1923 candidates
2. Python : 1882 candidates
3. AWS : 1388 candidates
4. Git : 1382 candidates
5. Docker : 1298 candidates
6. Linux : 1293 candidates
7. Kubernetes : 1271 candidates
8. Java : 1251 candidates
9. NLP : 561 candidates
10. TensorFlow : 554 candidates



KEY FINDINGS:

- Top 3 skills : SQL, Python, AWS
- Total unique skills : 210
- Implication : SQL, Python and AWS are universal – critical for JD matching

```
# — 4.4 Education & Profile Quality —
print("EDUCATION & PROFILE QUALITY ANALYSIS")
print("-" * 50)

edu_counts = df['highest_education'].value_counts()
tier_counts = df['institution_tier'].value_counts()

print("Education Level:")
for edu, cnt in edu_counts.items():
    print(f" {edu:<20}: {cnt} ({cnt/len(df)*100:.1f}%)")

print("\nInstitution Tier:")
for tier, cnt in tier_counts.items():
    print(f" {tier}: {cnt} ({cnt/len(df)*100:.1f}%)")

print(f"\nProfile Completeness: Mean = {df['profile_completeness_score'].mean():.2f} ({df['profile_completeness_score'].mean()*100:.1f}%)")

fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Pie chart – education level
axes[0].pie(edu_counts.values, labels=edu_counts.index,
            autopct='%1.1f%%', colors=palette[:len(edu_counts)],
            startangle=90, textprops={'fontsize': 11})
axes[0].set_title('Education Level Distribution', fontsize=13, fontweight='bold')

# Bar chart – institution tier
tierBars = axes[1].bar(tier_counts.index, tier_counts.values,
                      color=[palette[3], palette[4], palette[5]])
```

```

axes[1].set_title('Institution Tier Distribution', fontsize=13, fontweight='bold')
axes[1].set_xlabel('Tier')
axes[1].set_ylabel('Count')
for bar, val in zip(tier_bars, tier_counts.values):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 10,
                 str(val), ha='center', fontweight='bold')

# Histogram – profile completeness
axes[2].hist(df['profile_completeness_score'], bins=15,
            color=palette[1], edgecolor='white', alpha=0.85)
axes[2].axvline(df['profile_completeness_score'].mean(), color='red',
               linestyle='--', linewidth=2,
               label=f"Mean: {df['profile_completeness_score'].mean():.2f}")
axes[2].set_title('Profile Completeness Score Distribution',
                 fontsize=13, fontweight='bold')
axes[2].set_xlabel('Score (0=Empty, 1=Complete)')
axes[2].set_ylabel('Number of Candidates')
axes[2].legend(fontsize=11)

plt.tight_layout()
plt.savefig('eda_04_education_profile.png', dpi=150, bbox_inches='tight')
plt.show()

print()
print("KEY FINDINGS:")
print(f" Education is fairly evenly split across Masters, MBA, LLM, Bachelors")
print(f" Institution tiers are balanced – no single tier dominates")
print(f" Average profile completeness = {df['profile_completeness_score'].mean()*100:.1f}% – mostly well-filled")
print(f" Implication: Education tier and completeness will be useful features")

```

EDUCATION & PROFILE QUALITY ANALYSIS

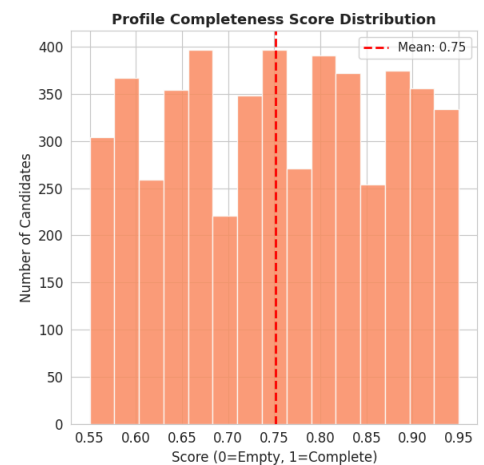
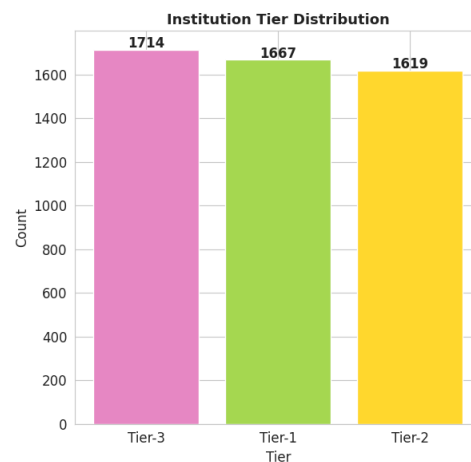
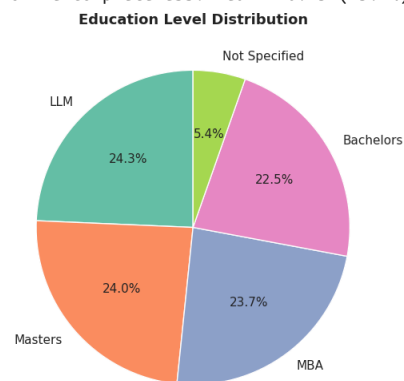
Education Level:

LLM	: 1216 (24.3%)
Masters	: 1202 (24.0%)
MBA	: 1185 (23.7%)
Bachelors	: 1127 (22.5%)
Not Specified	: 270 (5.4%)

Institution Tier:

Tier-3:	1714 (34.3%)
Tier-1:	1667 (33.3%)
Tier-2:	1619 (32.4%)

Profile Completeness: Mean = 0.75 (75.2%)



KEY FINDINGS:

Education is fairly evenly split across Masters, MBA, LLM, Bachelors
 Institution tiers are balanced – no single tier dominates
 Average profile completeness = 75.2% – mostly well-filled
 Implication: Education tier and completeness will be useful features

— 4.5 ATS Experience Flags Analysis —————
 # 16 binary flags indicate whether a candidate has specific experience

```

# 1 = Yes, has this experience | 0 = No
print("ATS EXPERIENCE FLAGS ANALYSIS")
print("-" * 50)

flag_cols = [c for c in df.columns if c.endswith('_flag')]
flag_labels = [c.replace('_flag', '').replace('_', ' ').title() for c in flag_cols]
flag_pct = (df[flag_cols].mean() * 100).values # % of candidates with each flag

flag_df = pd.DataFrame({'Experience Type': flag_labels, 'Candidates (%)': flag_pct})
flag_df = flag_df.sort_values('Candidates (%)', ascending=False)

print("Percentage of Candidates with Each ATS Flag:")
for _, row in flag_df.iterrows():
    bar = '█' * int(row['Candidates (%)'] / 5)
    print(f" {row['Experience Type']:<40}: {row['Candidates (%)']:5.1f}% {bar}")

# Color code: green=common (>60%), orange=moderate, red=rare (<30%)
colors = ['#2ecc71' if p > 60 else '#e74c3c' if p < 30 else '#f39c12'
          for p in flag_df['Candidates (%)']]

plt.figure(figsize=(14, 8))
bars = plt.barh(flag_df['Experience Type'], flag_df['Candidates (%)'], color=colors)
plt.title('ATS Experience Flags – % of Candidates\n(Green>60% | Orange=30-60% | Red<30%)',
          fontsize=14, fontweight='bold')
plt.xlabel('Percentage of Candidates with this Experience (%)', fontsize=12)
plt.axvline(50, color='navy', linestyle='--', linewidth=1.5, alpha=0.6, label='50% line')
for bar, val in zip(bars, flag_df['Candidates (%)']):
    plt.text(val + 0.5, bar.get_y() + bar.get_height()/2,
             f'{val:.1f}%', va='center', fontsize=9)
plt.legend(fontsize=11)
plt.tight_layout()
plt.savefig('eda_05_ats_flags.png', dpi=150, bbox_inches='tight')
plt.show()

print()
print("KEY FINDINGS:")
top_flag = flag_df.iloc[0]
bot_flag = flag_df.iloc[-1]
print(f" Most common flag : {top_flag['Experience Type']} ({top_flag['Candidates (%)']:5.1f}%)")
print(f" Rarest flag      : {bot_flag['Experience Type']} ({bot_flag['Candidates (%)']:5.1f}%)")
print(f" Implication        : All 16 flags will be used as ML features")

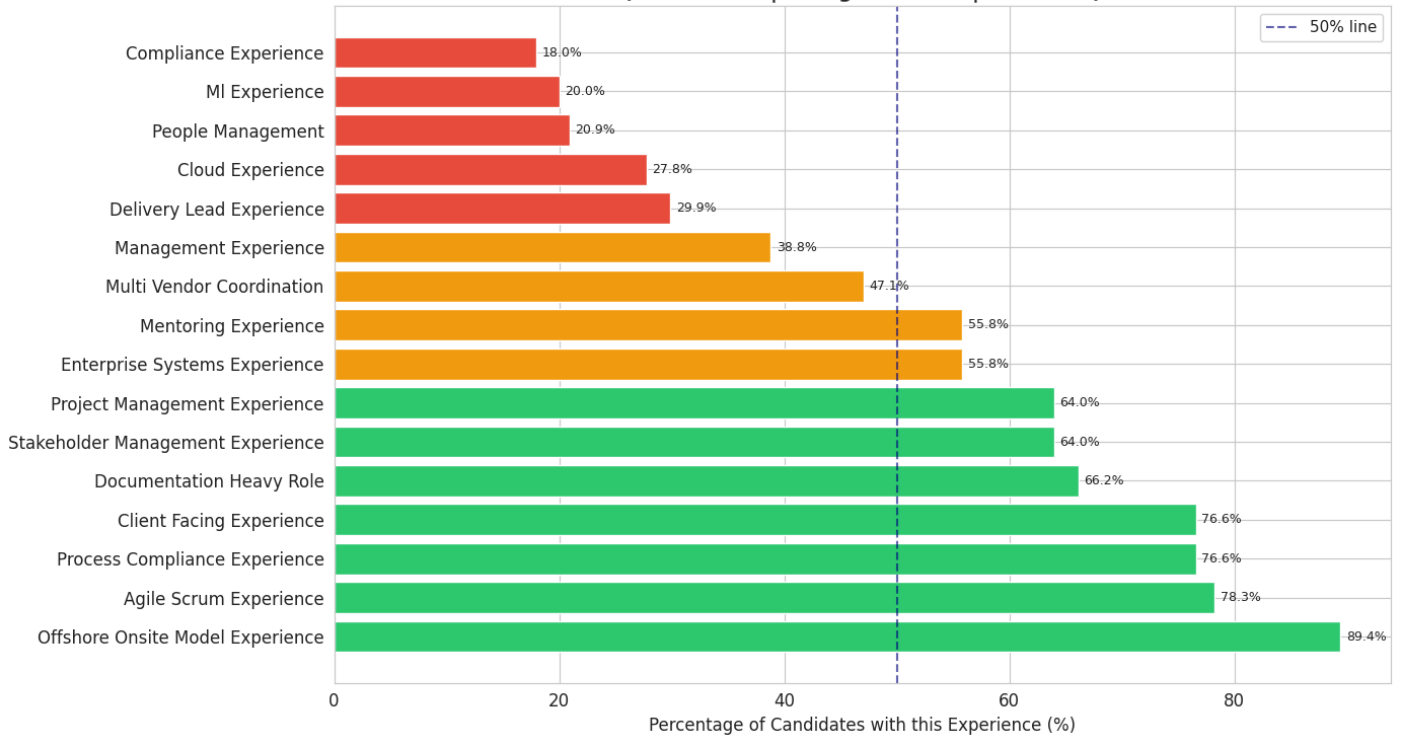
```

ATS EXPERIENCE FLAGS ANALYSIS

Percentage of Candidates with Each ATS Flag:

Offshore Onsite Model Experience	: 89.4%
Agile Scrum Experience	: 78.3%
Process Compliance Experience	: 76.6%
Client Facing Experience	: 76.6%
Documentation Heavy Role	: 66.2%
Stakeholder Management Experience	: 64.0%
Project Management Experience	: 64.0%
Enterprise Systems Experience	: 55.8%
Mentoring Experience	: 55.8%
Multi Vendor Coordination	: 47.1%
Management Experience	: 38.8%
Delivery Lead Experience	: 29.9%
Cloud Experience	: 27.8%
People Management	: 20.9%
ML Experience	: 20.0%
Compliance Experience	: 18.0%

ATS Experience Flags — % of Candidates
(Green>60% | Orange=30-60% | Red<30%)



KEY FINDINGS:

Most common flag : Offshore Onsite Model Experience (89.4%)
 Rarest flag : Compliance Experience (18.0%)
 Implication : All 16 flags will be used as ML features

Section 5: Feature Engineering

Why this step?

The raw dataset does not directly tell us **how well a candidate fits a job**. We must **engineer (create) new features** that quantify alignment:

Feature	What it Measures
<code>mandatory_match_pct</code>	% of must-have skills matched
<code>optional_match_pct</code>	% of nice-to-have skills matched
<code>skill_score</code>	Weighted skill alignment score
<code>experience_bucket</code>	Junior / Mid / Senior / Lead
<code>experience_score</code>	How well exp meets JD minimum
<code>domain_match</code>	Does candidate domain match JD?
<code>education_total_score</code>	Degree + institution tier bonus

leadership_score_pct	Leadership/management exposure
ats_composite_score	Overall ATS flag composite

These engineered features are the **backbone of our scoring model**.

```
# — 5.1 Define Job Descriptions for Target Roles —————
# We define what each role NEEDS – mandatory and optional skills
# This is the reference against which candidates will be scored

JOB_DESCRIPTIONS = {
    'Data Scientist': {
        'mandatory_skills': ['Python', 'Machine Learning', 'SQL', 'Statistics'],
        'optional_skills': ['TensorFlow', 'NLP', 'Power BI', 'Deep Learning'],
        'min_experience' : 3,
        'target_domain' : 'Data Science',
        'jd_text': 'python machine learning sql statistics tensorflow nlp pandas scikit learn data analysis model training feature engine'
    },
    'Senior Software Engineer': {
        'mandatory_skills': ['Python', 'Java', 'Git', 'Docker'],
        'optional_skills': ['Kubernetes', 'AWS', 'Linux', 'SQL'],
        'min_experience' : 5,
        'target_domain' : 'IT',
        'jd_text': 'python java git docker kubernetes aws linux software development distributed systems microservices backend api rest'
    },
    'Cloud Engineer': {
        'mandatory_skills': ['AWS', 'Docker', 'Kubernetes', 'Linux'],
        'optional_skills': ['Python', 'Git', 'SQL', 'Terraform'],
        'min_experience' : 3,
        'target_domain' : 'IT',
        'jd_text': 'aws docker kubernetes linux cloud infrastructure automation python devops deployment monitoring scalability reliability'
    },
    'ML Engineer': {
        'mandatory_skills': ['Python', 'Machine Learning', 'TensorFlow', 'SQL'],
        'optional_skills': ['NLP', 'Docker', 'AWS', 'Pandas'],
        'min_experience' : 2,
        'target_domain' : 'Data Science',
        'jd_text': 'python tensorflow machine learning sql nlp docker aws pandas model deployment mlops pipeline training inference'
    },
    'HR Business Partner': {
        'mandatory_skills': ['HRMS', 'Recruitment', 'Employee Engagement', 'HR Analytics'],
        'optional_skills': ['Payroll', 'Compliance', 'Training'],
        'min_experience' : 3,
        'target_domain' : 'HR',
        'jd_text': 'hrms recruitment employee engagement hr analytics payroll compliance training talent acquisition people management culture change'
    }
}

# — SELECT YOUR TARGET ROLE HERE —————
TARGET_ROLE = 'Data Scientist' # Change to any role above if needed
jd = JOB_DESCRIPTIONS[TARGET_ROLE]

print(f"Target Role Selected : {TARGET_ROLE}")
print(f"Mandatory Skills : {jd['mandatory_skills']}")
print(f"Optional Skills : {jd['optional_skills']}")
print(f"Min Experience Req : {jd['min_experience']} years")
print(f"Target Domain : {jd['target_domain']}")
print()
print(f"All available roles: {list(JOB_DESCRIPTIONS.keys())}")
```

```
Target Role Selected : Data Scientist
Mandatory Skills : ['Python', 'Machine Learning', 'SQL', 'Statistics']
Optional Skills : ['TensorFlow', 'NLP', 'Power BI', 'Deep Learning']
Min Experience Req : 3 years
Target Domain : Data Science
```

```
All available roles: ['Data Scientist', 'Senior Software Engineer', 'Cloud Engineer', 'ML Engineer', 'HR Business Partner']
```

```
# — 5.2 Skill Normalization & Match Scoring —————
# Normalize skill representations so 'AWS' and 'aws' are treated the same
```

```
SKILL_NORMALIZATION_MAP = {
    'ml' : 'Machine Learning',
    'machine learning' : 'Machine Learning',
    'deep learning' : 'Deep Learning',
    'nlp' : 'NLP',
    'natural language processing' : 'NLP',
    'python' : 'Python',
    'py' : 'Python',
    'ptyhon' : 'Python',
    'pythno' : 'Python',
    'sql' : 'SQL',
    'mysql' : 'SQL',
```

```

'postgresql' : 'SQL',
'aws' : 'AWS',
'amazon web services' : 'AWS',
'k8s' : 'Kubernetes',
'kubernetes' : 'Kubernetes',
'tensorflow' : 'TensorFlow',
'sklearn' : 'Scikit-Learn',
'scikit-learn' : 'Scikit-Learn',
'js' : 'JavaScript',
'javascript' : 'JavaScript',
}

def normalize_skills(text):
    """Convert raw skill text to a normalized list of skills"""
    if pd.isna(text): return []
    skills = [s.strip() for s in text.split(',') ] # Split by comma
    # Map each skill to its normalized form (or keep as-is if not in map)
    return [SKILL_NORMALIZATION_MAP.get(s.lower().strip(), s.strip()) for s in skills]

def compute_skill_match(candidate_skills, required_skills):
    """Calculate what % of required skills a candidate possesses"""
    if not required_skills: return 0.0
    candidate_lower = [s.lower() for s in candidate_skills] # Case-insensitive compare
    matched = sum(1 for s in required_skills if s.lower() in candidate_lower)
    return round(matched / len(required_skills) * 100, 1) # Return as percentage

# Build combined skills list (technical + tools)
df['skills_normalized'] = df['technical_skills_clean'].apply(normalize_skills)
df['tools_normalized'] = df['tools_platforms_raw'].apply(normalize_skills)
df['all_skills'] = df['skills_normalized'] + df['tools_normalized'] # Combine both

# Compute skill match percentages against JD requirements
df['mandatory_match_pct'] = df['all_skills'].apply(
    lambda s: compute_skill_match(s, jd['mandatory_skills']))
df['optional_match_pct'] = df['all_skills'].apply(
    lambda s: compute_skill_match(s, jd['optional_skills']))

# Weighted skill score: mandatory skills matter more (70%) than optional (30%)
df['skill_score'] = (df['mandatory_match_pct'] * 0.70 +
    df['optional_match_pct'] * 0.30).round(1)

print("SKILL MATCH RESULTS (Sample):")
print(df[['candidate_name', 'mandatory_match_pct', 'optional_match_pct', 'skill_score']].head(8).to_string(index=False))
print()
print(f"Average mandatory skill match : {df['mandatory_match_pct'].mean():.1f}%")
print(f"Average optional skill match : {df['optional_match_pct'].mean():.1f}%")
print(f"Candidates with 100% mandatory match: {(df['mandatory_match_pct']==100).sum()}")

```

```

SKILL MATCH RESULTS (Sample):
candidate_name  mandatory_match_pct  optional_match_pct  skill_score
Candidate_1      50.0                25.0             42.5
Candidate_2      25.0                0.0             17.5
Candidate_3       0.0                0.0              0.0
Candidate_4      25.0                0.0             17.5
Candidate_5       0.0                0.0              0.0
Candidate_6      75.0                25.0             60.0
Candidate_7      25.0                0.0             17.5
Candidate_8       0.0                0.0              0.0

```

```

Average mandatory skill match : 21.6%
Average optional skill match : 8.3%
Candidates with 100% mandatory match: 0

```

— 5.3 Experience & Seniority Features —

```

def categorize_experience(years):
    """Bucket continuous years into seniority levels"""
    if years <= 2 : return 'Junior' # 0-2 years
    elif years <= 5: return 'Mid'   # 3-5 years
    elif years <= 10: return 'Senior' # 6-10 years
    else : return 'Lead' # 10+ years

df['experience_bucket'] = df['years_experience'].apply(categorize_experience)

# Experience score: how well does candidate meet the JD minimum requirement?
# If JD needs 3 years and candidate has 6 → score = min(100, 6/3*100) = 100
# If JD needs 3 years and candidate has 1 → score = min(100, 1/3*100) = 33.3
df['experience_score'] = df['years_experience'].apply(
    lambda y: min(100, round(y / jd['min_experience'] * 100, 1)))

# Seniority alignment: does experience level match what JD expects?
df['seniority_alignment'] = (
    (df['experience_bucket'] == 'Senior') |
    (df['experience_bucket'] == 'Mid')
).astype(int) # 1=aligned, 0=not aligned

```



```
# — 5.4 Domain Alignment Feature —————
# Does the candidate's primary domain match the job's domain?
df['domain_match'] = (df['primary_domain'] == jd['target_domain']).astype(int)
df['domain_match_pct'] = df['domain_match'] * 100 # Convert to 0/100 scale

# — 5.5 Education Score —————
# Higher degrees and better institutions get higher scores
edu_score_map = {'Masters': 100, 'MBA': 90, 'LLM': 80, 'Bachelors': 70, 'Not Specified': 50}
tier_bonus_map = {'Tier-1': 10, 'Tier-2': 5, 'Tier-3': 0} # Tier-1 gets bonus points

df['education_score'] = df['highest_education'].map(edu_score_map).fillna(50)
df['institution_bonus'] = df['institution_tier'].map(tier_bonus_map).fillna(0)
df['education_total_score'] = (df['education_score'] + df['institution_bonus']).clip(upper=100)

print("EXPERIENCE BUCKETING RESULTS:")
print(df['experience_bucket'].value_counts().to_string())
print()
print(f"Domain match rate      : {df['domain_match'].mean()*100:.1f}% of candidates match '{jd['target_domain']}'")
print(f"Avg experience score   : {df['experience_score'].mean():.1f} / 100")
print(f"Avg education score    : {df['education_total_score'].mean():.1f} / 100")
```

```
EXPERIENCE BUCKETING RESULTS:
experience_bucket
Senior      2259
Mid         1680
Lead        533
Junior      528

Domain match rate      : 20.0% of candidates match 'Data Science'
Avg experience score   : 94.8 / 100
Avg education score    : 87.0 / 100
```

```
# — 5.6 Leadership & ATS Composite Features —————

# Leadership composite – average of 5 leadership-related flags
leadership_flags = [
    'management_experience_flag',
    'people_management_flag',
    'delivery_lead_experience_flag',
    'mentoring_experience_flag',
    'stakeholder_management_experience_flag'
]
df['leadership_score_pct'] = (
    df[leadership_flags].sum(axis=1) / len(leadership_flags) * 100).round(1)

# ATS composite – average of ALL 16 experience flags
all_flags = [c for c in df.columns if c.endswith('_flag')]
df['ats_composite_score'] = (
    df[all_flags].sum(axis=1) / len(all_flags) * 100).round(1)
```

```
# — 5.7 RULE-BASED FIT SCORE —————
# Combine all features with business-logic weights:
# Skills match = 40% (most critical for role alignment)
# Experience   = 25% (seniority matters a lot)
# Domain       = 15% (relevant industry background)
# Education    = 10% (academic foundation)
# ATS flags    = 10% (soft experience indicators)
df['rule_fit_score'] = (
    df['skill_score'] * 0.40 +
    df['experience_score'] * 0.25 +
    df['domain_match_pct'] * 0.15 +
    df['education_total_score'] * 0.10 +
    df['ats_composite_score'] * 0.10
).clip(0, 100).round(2)
```

```
# — 5.8 Fit Label (Target Variable) —————
# We create our own label using business logic (no pre-labeled data available)
# This label will be used to train the ML classifier
def assign_fit_label(score):
    """Convert numeric score to business-friendly fit category"""
    if score >= 70 : return 'High Fit'    # Strong candidate – shortlist
    elif score >= 45: return 'Medium Fit' # Moderate fit – human review
    else           : return 'Low Fit'     # Poor fit – deprioritize

df['fit_label'] = df['rule_fit_score'].apply(assign_fit_label)

print("FIT LABEL DISTRIBUTION (Target Variable):")
for label, count in df['fit_label'].value_counts().items():
    pct = count / len(df) * 100
    print(f" {label:<15}: {count:>4} candidates ({pct:.1f}%)")
print()
print(f"Rule-Based Fit Score Stats:")
print(f" Min   : {df['rule_fit_score'].min():.1f}")
print(f" Max   : {df['rule_fit_score'].max():.1f}")
```

```
print(f" Mean : {df['rule_fit_score'].mean():.2f}")
print()
print("FEATURE ENGINEERING COMPLETE!")
print(f"Total features in dataset: {df.shape[1]} (original 34 + {df.shape[1]-34} engineered)")
```

```
FIT LABEL DISTRIBUTION (Target Variable):
Low Fit      : 2436 candidates (48.7%)
Medium Fit   : 2116 candidates (42.3%)
High Fit     : 448 candidates (9.0%)
```

```
Rule-Based Fit Score Stats:
Min   : 13.9
Max   : 82.1
Mean  : 47.64
```

```
FEATURE ENGINEERING COMPLETE!
Total features in dataset: 53 (original 34 + 19 engineered)
```

Section 6: NLP — TF-IDF Text Similarity Scoring

Why this step?

Rule-based scoring only checks **exact skill keyword matches**. But a candidate's **experience summary and project descriptions** contain rich contextual information that pure rules cannot capture.

TF-IDF (Term Frequency–Inverse Document Frequency) converts text into numerical vectors. **Cosine similarity** then measures how similar a candidate's resume text is to the job description text.

- **High similarity** → Resume language closely matches JD language → Strong fit
- **Low similarity** → Resume language diverges from JD → Weak fit

Hybrid Score = 60% Rule-Based + 40% NLP Similarity

This combination gives us the best of both worlds: structured feature matching + contextual NLP understanding.

```
# — 6.1 Text Preprocessing —————
# Clean resume text before vectorization:
# 1. Lowercase → standardize case
# 2. Remove punctuation → keep only letters
# 3. Tokenize → split into words
# 4. Remove stopwords → remove 'the', 'is', 'and' etc.
# 5. Lemmatize → 'running' → 'run', 'machines' → 'machine'

def preprocess_text(text):
    """Clean and normalize text for TF-IDF vectorization"""
    if pd.isna(text) or text.strip() == '': return ''
    text = str(text).lower()                # Step 1: Lowercase
    text = re.sub(r'[^\a-z\s]', ' ', text)  # Step 2: Remove non-letters
    try:
        tokens = word_tokenize(text)        # Step 3: Tokenize into words
    except:
        tokens = text.split()
    tokens = [lemmatizer.lemmatize(t) for t in tokens] # Step 5: Lemmatize
    tokens = [t for t in tokens if t not in stop_words and len(t) > 2] # Step 4: Remove stopwords
    return ' '.join(tokens)                # Join back into string

# Combine multiple resume text fields into one rich text representation
df['resume_text'] = (
    df['technical_skills_clean'].fillna('') + ' ' + # Skills
    df['experience_summary'].fillna('') + ' ' + # Experience description
    df['project_summary'].fillna('') + ' ' + # Project descriptions
    df['key_achievements'].fillna('') + # Achievements
)

# Apply preprocessing
df['resume_text_clean'] = df['resume_text'].apply(preprocess_text)

print("TEXT PREPROCESSING COMPLETE!")
print(f"Example – Original text:")
print(f"  '{df['resume_text'].iloc[0][:100]}...")
print()
print(f"Example – After preprocessing:")
print(f"  '{df['resume_text_clean'].iloc[0][:100]}...")
print()
print(f"Avg words per resume (before): {df['resume_text'].str.split().str.len().mean():.0f}")
print(f"Avg words per resume (after) : {df['resume_text_clean'].str.split().str.len().mean():.0f}")

TEXT PREPROCESSING COMPLETE!
Example – Original text:
'Machine Learning, Power BI, Pandas, Python Hands-on data science professional with 7+ years of end-t...
```

```
Example - After preprocessing:
'machine learning power panda python hand data science professional year end end delivery experience ...'
```

```
Avg words per resume (before): 50
Avg words per resume (after) : 35
```

— 6.2 TF-IDF Vectorization & Cosine Similarity —

```
# Preprocess JD text using the same pipeline
jd_text_clean = preprocess_text(jd['jd_text'])
print(f"Preprocessed JD text: '{jd_text_clean}'")
print()

# TF-IDF Vectorizer settings:
# max_features=1000 : Keep only top 1000 terms
# ngram_range=(1,2) : Use single words AND word pairs (bigrams)
# min_df=2 : Ignore terms appearing in < 2 documents
vectorizer = TfidfVectorizer(
    max_features=1000,
    ngram_range=(1, 2),
    min_df=2
)

# Fit vectorizer on ALL texts (resumes + JD) together
all_texts = df['resume_text_clean'].tolist() + [jd_text_clean]
tfidf_matrix = vectorizer.fit_transform(all_texts)

# Separate resume matrix and JD vector
resume_matrix = tfidf_matrix[:-1] # All rows except last = resumes
jd_vector = tfidf_matrix[-1] # Last row = JD

# Compute cosine similarity: 0=no match, 1=perfect match
similarity_scores = cosine_similarity(resume_matrix, jd_vector).flatten()
df['tfidf_similarity'] = (similarity_scores * 100).round(2) # Convert to 0-100 scale

# Normalize to 0-100 scale relative to the best match
tfidf_max = df['tfidf_similarity'].max()
df['tfidf_normalized'] = (
    df['tfidf_similarity'] / tfidf_max * 100).round(2) if tfidf_max > 0 else 0

print("TF-IDF SIMILARITY RESULTS:")
print(f" Vocabulary size : {len(vectorizer.vocabulary_):,} terms")
print(f" Min similarity : {df['tfidf_similarity'].min():.2f}%")
print(f" Max similarity : {df['tfidf_similarity'].max():.2f}%")
print(f" Mean similarity : {df['tfidf_similarity'].mean():.2f}%")
print()
print("Top 5 Candidates by NLP Similarity:")
top_nlp = df.nlargest(5, 'tfidf_similarity')[['candidate_name', 'primary_role', 'tfidf_similarity']]
print(top_nlp.to_string(index=False))
```

Preprocessed JD text: 'python machine learning sql statistic tensorflow nlp panda scikit learn data analysis model training feature engine'

```
TF-IDF SIMILARITY RESULTS:
Vocabulary size : 1,000 terms
Min similarity : 0.00%
Max similarity : 37.59%
Mean similarity : 5.10%
```

```
Top 5 Candidates by NLP Similarity:
candidate_name primary_role tfidf_similarity
Candidate_3131 Junior Data Scientist 37.59
Candidate_4671 Data Scientist 36.64
Candidate_3825 Data Analyst 36.52
Candidate_4802 Data Scientist 36.51
Candidate_4938 Data Analyst 36.45
```

— 6.3 Compute Hybrid Fit Score —

```
# Hybrid = Rule-Based Score (60%) + NLP Similarity Score (40%)
# This balances structured feature matching with contextual NLP understanding

df['hybrid_fit_score'] = (
    df['rule_fit_score'] * 0.60 + # Rule-based (skills, exp, domain, edu, ATS)
    df['tfidf_normalized'] * 0.40 # NLP text similarity
).clip(0, 100).round(2)

# Assign final shortlisting recommendation
df['hybrid_fit_label'] = df['hybrid_fit_score'].apply(
    lambda s: 'High Fit' if s >= 65 else
              'Medium Fit' if s >= 40 else
              'Low Fit'
)

print("HYBRID FIT SCORE DISTRIBUTION:")
for label, count in df['hybrid_fit_label'].value_counts().items():
```

```

pct = count / len(df) * 100
print(f"  {label:<15}: {count:>4} ({pct:.1f}%)"
print()
print(f"Score range : {df['hybrid_fit_score'].min():.1f} - {df['hybrid_fit_score'].max():.1f}")
print(f"Mean score  : {df['hybrid_fit_score'].mean():.2f}")

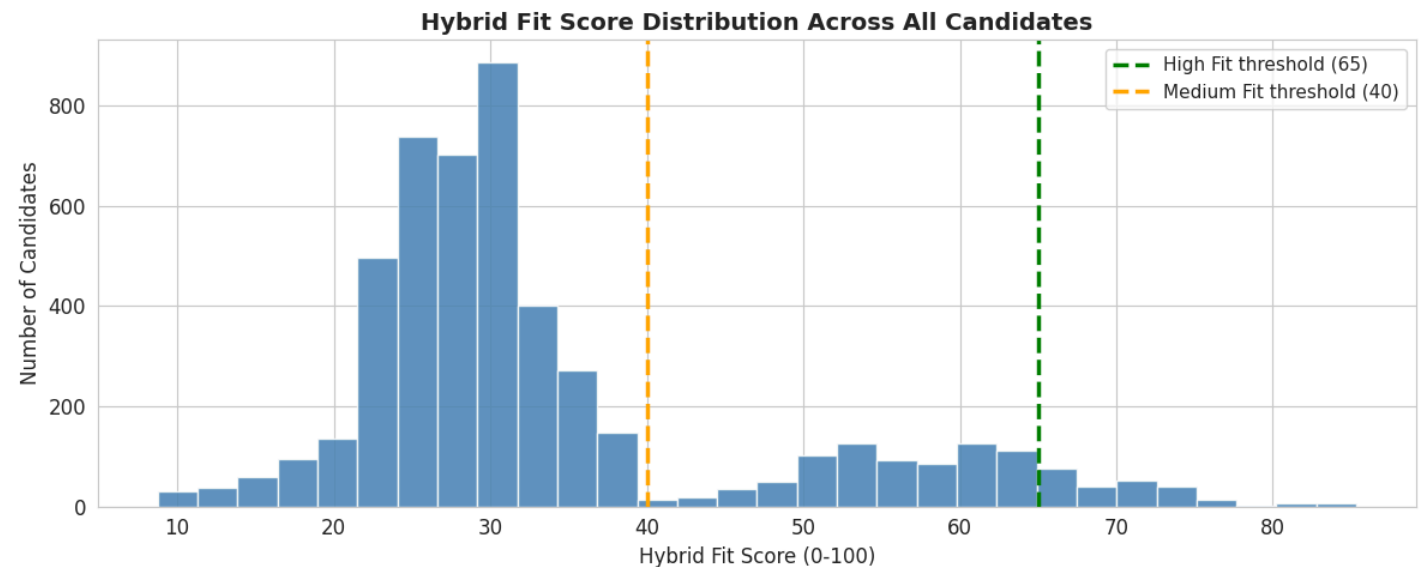
# Visualize score distribution
plt.figure(figsize=(12, 5))
plt.hist(df['hybrid_fit_score'], bins=30, color='steelblue', edgecolor='white', alpha=0.85)
plt.axvline(65, color='green', linestyle='--', lw=2.5, label='High Fit threshold (65)')
plt.axvline(40, color='orange', linestyle='--', lw=2.5, label='Medium Fit threshold (40)')
plt.title('Hybrid Fit Score Distribution Across All Candidates',
        fontsize=14, fontweight='bold')
plt.xlabel('Hybrid Fit Score (0-100)')
plt.ylabel('Number of Candidates')
plt.legend(fontsize=11)
plt.tight_layout()
plt.savefig('nlp_hybrid_score.png', dpi=150, bbox_inches='tight')
plt.show()
print("Interpretation: Most candidates cluster in Medium Fit range – expected for diverse pool")

```

HYBRID FIT SCORE DISTRIBUTION:

Low Fit	: 4004 (80.1%)
Medium Fit	: 758 (15.2%)
High Fit	: 238 (4.8%)

Score range : 8.8 - 85.3
Mean score : 34.02



Interpretation: Most candidates cluster in Medium Fit range – expected for diverse pool

Section 7: ML Modeling & Evaluation

Why this step?

We train ML classifiers to **predict fit labels** (High/Medium/Low) from features. This allows the system to **learn patterns** and generalize to new resumes.

Models Used:

Model	Why Chosen
Logistic Regression	Simple, interpretable baseline
Random Forest	Handles non-linear patterns, robust
Gradient Boosting	High accuracy on tabular data

Evaluation Metrics:

- **Accuracy** — Overall % correctly classified
- **Precision** — Of predicted High Fit, how many truly are?
- **Recall** — Of actual High Fit, how many did we catch?
- **F1-Score** — Balance of Precision + Recall
- **Confusion Matrix** — Visual breakdown of correct/wrong predictions

```
# — 7.1 Prepare Features & Target —————

# Select all engineered features for the model
feature_cols = [
    # Original numeric features
    'years_experience', 'keyword_density_score', 'profile_completeness_score',
    'resume_length_words',
    # Engineered skill features
    'mandatory_match_pct', 'optional_match_pct', 'skill_score',
    # Engineered experience features
    'experience_score', 'seniority_alignment',
    # Engineered domain & education features
    'domain_match', 'education_score', 'institution_bonus',
    # Composite scores
    'leadership_score_pct', 'ats_composite_score',
    # NLP similarity
    'tfidf_normalized',
    # Key ATS binary flags
    'management_experience_flag', 'project_management_experience_flag',
    'agile_scrum_experience_flag', 'client_facing_experience_flag',
    'cloud_experience_flag', 'ml_experience_flag',
    'stakeholder_management_experience_flag'
]

# Encode target label: High Fit=0, Low Fit=1, Medium Fit=2 (alphabetical)
le = LabelEncoder()
y = le.fit_transform(df['hybrid_fit_label'])
X = df[feature_cols].fillna(0) # Fill any remaining NaN with 0

# Split: 80% training, 20% testing
# stratify=y ensures proportional class distribution in both sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

print("TRAIN-TEST SPLIT:")
print(f" Training set : {len(X_train):,} samples (80%)")
print(f" Testing set  : {len(X_test):,} samples (20%)")
print(f" Features used : {len(feature_cols)}")
print(f" Target classes: {list(le.classes_)}")
print()
print("Class distribution in training set:")
for cls, label in zip([0,1,2], le.classes_):
    count = (y_train == cls).sum()
    print(f" {label:<15}: {count} ({count/len(y_train)*100:.1f}%)")
```

```
TRAIN-TEST SPLIT:
 Training set : 4,000 samples (80%)
 Testing set  : 1,000 samples (20%)
 Features used : 22
 Target classes: ['High Fit', 'Low Fit', 'Medium Fit']
```

```
Class distribution in training set:
 High Fit      : 190 (4.8%)
 Low Fit       : 3203 (80.1%)
 Medium Fit    : 607 (15.2%)
```

— 7.2 Train Multiple Models —————

```
# Scale features for Logistic Regression (tree models don't need scaling)
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

# Define all models to train
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Random Forest'      : RandomForestClassifier(n_estimators=100, random_state=42),
    'Gradient Boosting'   : GradientBoostingClassifier(n_estimators=100, random_state=42)
}

print("TRAINING MODELS...")
print("-" * 50)

results = {}
trained = {}

for name, model in models.items():
    # Logistic Regression uses scaled data; tree models use raw
    X_tr = X_train_sc if 'Logistic' in name else X_train
    X_te = X_test_sc if 'Logistic' in name else X_test

    model.fit(X_tr, y_train) # Train model
    preds = model.predict(X_te) # Predict on test set
    acc = accuracy_score(y_test, preds)
```

```

results[name] = acc
trained[name] = (model, preds)
print(f" {name:<28}: Accuracy = {acc*100:.2f}%")

# Identify best performing model
best_name = max(results, key=results.get)
best_model, best_preds = trained[best_name]

print()
print(f"BEST MODEL: {best_name} (Accuracy: {results[best_name]*100:.2f}%")

```

TRAINING MODELS...

```

-----
Logistic Regression      : Accuracy = 99.00%
Random Forest           : Accuracy = 99.00%
Gradient Boosting       : Accuracy = 98.80%

```

BEST MODEL: Logistic Regression (Accuracy: 99.00%)

— 7.3 Best Model – Detailed Evaluation —————

```

print(f"DETAILED EVALUATION: {best_name}")
print("=" * 60)
print()
print("CLASSIFICATION REPORT:")
print("(precision = accuracy for that label)")
print("(recall    = coverage – did we catch all of them?)")
print("(f1-score   = balance of precision and recall)")
print()
print(classification_report(y_test, best_preds, target_names=le.classes_))

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Confusion Matrix
cm = confusion_matrix(y_test, best_preds)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', linewidths=0.5,
            xticklabels=le.classes_, yticklabels=le.classes_, ax=axes[0],
            annot_kws={'size': 14, 'weight': 'bold'})
axes[0].set_title(f'Confusion Matrix\n{best_name}', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Actual Label', fontsize=12)
axes[0].set_xlabel('Predicted Label', fontsize=12)

# Model comparison bar chart
model_names = list(results.keys())
accuracies = [v * 100 for v in results.values()]
bar_colors = ['#e74c3c', '#3498db', '#2ecc71']
bars = axes[1].bar(model_names, accuracies, color=bar_colors, edgecolor='white')
axes[1].set_title('Model Accuracy Comparison', fontsize=13, fontweight='bold')
axes[1].set_ylabel('Accuracy (%)', fontsize=12)
axes[1].set_ylim(0, 110)
for bar, acc in zip(bars, accuracies):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1.5,
                 f'{acc:.1f}%', ha='center', fontsize=12, fontweight='bold')

plt.tight_layout()
plt.savefig('model_evaluation.png', dpi=150, bbox_inches='tight')
plt.show()

print("INTERPRETATION OF CONFUSION MATRIX:")
print(" Diagonal cells (top-left to bottom-right) = Correct predictions")
print(" Off-diagonal cells = Misclassifications")
print(" Goal: Maximize diagonal, minimize off-diagonal values")

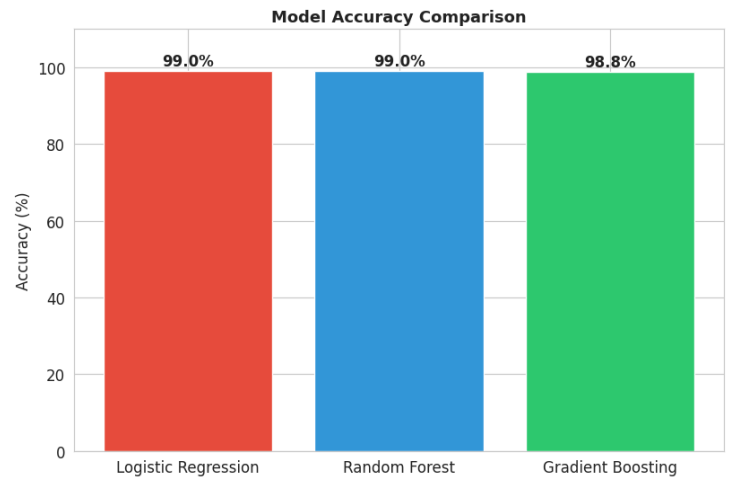
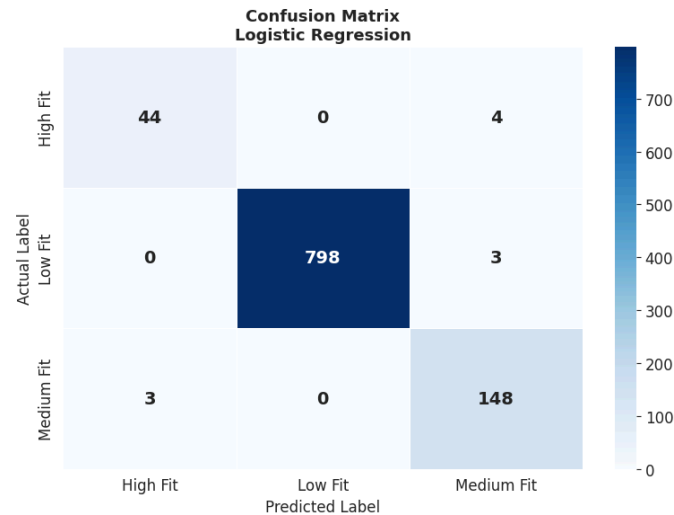
```

DETAILED EVALUATION: Logistic Regression

CLASSIFICATION REPORT:

(precision = accuracy for that label)
 (recall = coverage – did we catch all of them?)
 (f1-score = balance of precision and recall)

	precision	recall	f1-score	support
High Fit	0.94	0.92	0.93	48
Low Fit	1.00	1.00	1.00	801
Medium Fit	0.95	0.98	0.97	151
accuracy			0.99	1000
macro avg	0.96	0.96	0.96	1000
weighted avg	0.99	0.99	0.99	1000



INTERPRETATION OF CONFUSION MATRIX:

Diagonal cells (top-left to bottom-right) = Correct predictions
 Off-diagonal cells = Misclassifications
 Goal: Maximize diagonal, minimize off-diagonal values

```
# — 7.4 Feature Importance Analysis —————
# Tree-based models can tell us which features matter most
# This helps recruiters understand WHAT drives fit scores

if hasattr(best_model, 'feature_importances_'):
    fi_df = pd.DataFrame({
        'Feature' : feature_cols,
        'Importance': best_model.feature_importances_
    }).sort_values('Importance', ascending=True)

    top15 = fi_df.tail(15)

    plt.figure(figsize=(12, 8))
    colors = ['#e74c3c' if imp > fi_df['Importance'].quantile(0.75)
              else '#3498db' for imp in top15['Importance']]
    plt.barh(top15['Feature'], top15['Importance'], color=colors)
    plt.title(f'Top 15 Feature Importances\n{best_name}', fontsize=14, fontweight='bold')
    plt.xlabel('Feature Importance Score', fontsize=12)
    plt.tight_layout()
    plt.savefig('feature_importance.png', dpi=150, bbox_inches='tight')
    plt.show()

    print("TOP 10 MOST IMPORTANT FEATURES:")
    for i, (_, row) in enumerate(fi_df.tail(10).iloc[::-1].iterrows(), 1):
        print(f" {i:2}. {row['Feature']:<40}: {row['Importance']:.4f}")
    print()
    print("INTERPRETATION:")
    top_feat = fi_df.iloc[-1]
    print(f" Most important feature: '{top_feat['Feature']}'")
    print(" This means this feature has the strongest influence on fit prediction")
    print(" Recruiters should focus most on this factor when evaluating candidates")
```

```
else:
    print("Feature importance not available for Logistic Regression.")
    print("Use Random Forest or Gradient Boosting for importance plots.")
```

```
Feature importance not available for Logistic Regression.
Use Random Forest or Gradient Boosting for importance plots.
```

Section 8: Final Rankings & Business Recommendations

Why this step?

The model's job is not just to produce scores — it must produce **actionable, explainable shortlisting recommendations** for recruiters.

Each candidate gets:

- A **final fit score** (0–100)
- A **shortlist recommendation** (High / Medium / Low Fit)
- A **plain-English explanation** of WHY they scored that way

This explainability is critical for recruiter trust and audit compliance.

```
# — 8.1 Generate Candidate Rankings —————

def generate_explanation(row):
    """Generate human-readable explanation for each candidate's fit score"""
    parts = []
    mand = row.get('mandatory_match_pct', 0)
    yrs = row.get('years_experience', 0)
    tfidf = row.get('tfidf_normalized', 0)

    # Explain skill match
    if mand >= 75:
        parts.append(f"Skills: {mand:.0f}% mandatory match [STRONG]")
    elif mand >= 50:
        parts.append(f"Skills: {mand:.0f}% mandatory match [PARTIAL]")
    else:
        parts.append(f"Skills: {mand:.0f}% mandatory match [WEAK]")

    # Explain experience
    if yrs >= jd['min_experience']:
        parts.append(f"Exp: {yrs}yrs [MEETS REQUIREMENT]")
    else:
        parts.append(f"Exp: {yrs}yrs [BELOW {jd['min_experience']}yr REQUIREMENT]")

    # Explain domain match
    parts.append('Domain: MATCH' if row.get('domain_match', 0) == 1 else 'Domain: MISMATCH')

    # Explain NLP similarity
    if tfidf >= 50:
        parts.append(f"NLP Similarity: {tfidf:.1f}% [HIGH]")
    elif tfidf >= 25:
        parts.append(f"NLP Similarity: {tfidf:.1f}% [MODERATE]")
    else:
        parts.append(f"NLP Similarity: {tfidf:.1f}% [LOW]")

    return ' | '.join(parts)

# Sort candidates by hybrid fit score (descending)
ranked = df.sort_values('hybrid_fit_score', ascending=False).reset_index(drop=True)
ranked['rank'] = ranked.index + 1 # Assign rank 1, 2, 3...
ranked['explanation'] = ranked.apply(generate_explanation, axis=1)

print(f"TOP 20 RANKED CANDIDATES FOR: {TARGET_ROLE}")
print("=" * 95)
display_cols = ['rank', 'candidate_name', 'years_experience',
                'mandatory_match_pct', 'hybrid_fit_score', 'hybrid_fit_label']
print(ranked[display_cols].head(20).to_string(index=False))
```

```
TOP 20 RANKED CANDIDATES FOR: Data Scientist
=====
rank candidate_name  years_experience  mandatory_match_pct  hybrid_fit_score  hybrid_fit_label
1 Candidate_1732      9              75.0              85.33          High Fit
2 Candidate_2421      8              75.0              83.96          High Fit
3 Candidate_1445      5              75.0              83.41          High Fit
4 Candidate_3825      7              50.0              83.41          High Fit
5 Candidate_4802      9              50.0              82.95          High Fit
6 Candidate_4671     10              50.0              82.86          High Fit
7 Candidate_3253      3              75.0              82.84          High Fit
8 Candidate_4938      7              50.0              82.74          High Fit
9 Candidate_225       10              75.0              82.22          High Fit
10 Candidate_1061     4              50.0              81.84          High Fit
```


11	Candidate_598	10	25.0	81.38	High Fit
12	Candidate_1715	4	50.0	81.06	High Fit
13	Candidate_326	5	50.0	80.76	High Fit
14	Candidate_747	3	75.0	80.37	High Fit
15	Candidate_640	3	50.0	78.94	High Fit
16	Candidate_3131	3	25.0	78.93	High Fit
17	Candidate_2465	9	75.0	77.36	High Fit
18	Candidate_4403	10	75.0	77.10	High Fit
19	Candidate_2821	9	75.0	77.08	High Fit
20	Candidate_3719	10	50.0	76.75	High Fit

— 8.2 Top 10 Candidates with Full Explanations —

```
print(f"TOP 10 CANDIDATES – DETAILED SHORTLISTING REPORT")
print(f"Role: {TARGET_ROLE} | Powered by NLP Hybrid Scoring")
print(f"=" * 90)

for _, row in ranked.head(10).iterrows():
    print()
    print(f"  Rank #{int(row['rank'])} | {row['candidate_name']}")
    print(f"  Role: {row['primary_role']} | Domain: {row['primary_domain']} | Exp: {row['years_experience']} yrs")
    print(f"  Final Fit Score : {row['hybrid_fit_score']:.1f}/100")
    print(f"  Recommendation  : {row['hybrid_fit_label']}")
    print(f"  Explanation     : {row['explanation']}")
    print(f"  {'-'*85}")

Recommendation : High Fit
Explanation    : Skills: 75% mandatory match [STRONG] | Exp: 8yrs [MEETS REQUIREMENT] | Domain: MATCH | NLP Similarity: 91.6% [HIGH]

Rank #3 | Candidate_1445
Role: Data Analyst | Domain: Data Science | Exp: 5 yrs
Final Fit Score : 83.4/100
Recommendation : High Fit
Explanation    : Skills: 75% mandatory match [STRONG] | Exp: 5yrs [MEETS REQUIREMENT] | Domain: MATCH | NLP Similarity: 94.0% [HIGH]

Rank #4 | Candidate_3825
Role: Data Analyst | Domain: Data Science | Exp: 7 yrs
Final Fit Score : 83.4/100
Recommendation : High Fit
Explanation    : Skills: 50% mandatory match [PARTIAL] | Exp: 7yrs [MEETS REQUIREMENT] | Domain: MATCH | NLP Similarity: 97.2% [HIGH]

Rank #5 | Candidate_4802
Role: Data Scientist | Domain: Data Science | Exp: 9 yrs
Final Fit Score : 83.0/100
Recommendation : High Fit
Explanation    : Skills: 50% mandatory match [PARTIAL] | Exp: 9yrs [MEETS REQUIREMENT] | Domain: MATCH | NLP Similarity: 97.1% [HIGH]

Rank #6 | Candidate_4671
Role: Data Scientist | Domain: Data Science | Exp: 10 yrs
Final Fit Score : 82.9/100
Recommendation : High Fit
Explanation    : Skills: 50% mandatory match [PARTIAL] | Exp: 10yrs [MEETS REQUIREMENT] | Domain: MATCH | NLP Similarity: 97.5% [HIGH]

Rank #7 | Candidate_3253
Role: ML Engineer | Domain: Data Science | Exp: 3 yrs
Final Fit Score : 82.8/100
Recommendation : High Fit
Explanation    : Skills: 75% mandatory match [STRONG] | Exp: 3yrs [MEETS REQUIREMENT] | Domain: MATCH | NLP Similarity: 93.3% [HIGH]

Rank #8 | Candidate_4938
Role: Data Analyst | Domain: Data Science | Exp: 7 yrs
Final Fit Score : 82.7/100
Recommendation : High Fit
Explanation    : Skills: 50% mandatory match [PARTIAL] | Exp: 7yrs [MEETS REQUIREMENT] | Domain: MATCH | NLP Similarity: 97.0% [HIGH]

Rank #9 | Candidate_225
Role: Data Scientist | Domain: Data Science | Exp: 10 yrs
Final Fit Score : 82.2/100
Recommendation : High Fit
Explanation    : Skills: 75% mandatory match [STRONG] | Exp: 10yrs [MEETS REQUIREMENT] | Domain: MATCH | NLP Similarity: 82.4% [HIGH]

Rank #10 | Candidate_1061
Role: Data Scientist | Domain: Data Science | Exp: 4 yrs
Final Fit Score : 81.8/100
Recommendation : High Fit
Explanation    : Skills: 50% mandatory match [PARTIAL] | Exp: 4yrs [MEETS REQUIREMENT] | Domain: MATCH | NLP Similarity: 96.4% [HIGH]
```

— 8.3 Final Summary Visualizations —

```
fig, axes = plt.subplots(1, 3, figsize=(20, 7))

# Chart 1: Score distribution with thresholds
```

```

axes[0].hist(df['hybrid_fit_score'], bins=30, color='steelblue',
            edgecolor='white', alpha=0.85)
axes[0].axvline(65, color='green', linestyle='--', lw=2.5,
              label='High Fit (≥65)')
axes[0].axvline(40, color='orange', linestyle='--', lw=2.5,
              label='Medium Fit (≥40)')
axes[0].fill_betweenx([0, axes[0].get_ylim()[1] if axes[0].get_ylim()[1] > 0 else 500],
                    65, 100, alpha=0.1, color='green')
axes[0].set_title('Hybrid Fit Score Distribution', fontsize=13, fontweight='bold')
axes[0].set_xlabel('Score (0-100)')
axes[0].set_ylabel('Number of Candidates')
axes[0].legend(fontsize=10)

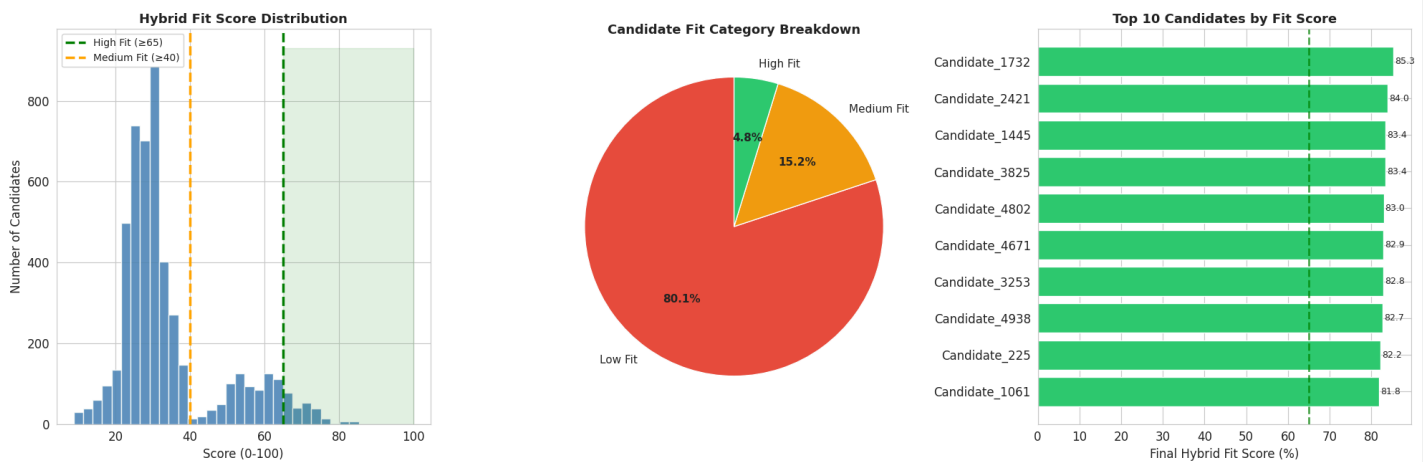
# Chart 2: Fit label pie chart
label_counts = df['hybrid_fit_label'].value_counts()
color_map = {'High Fit': '#2ecc71', 'Medium Fit': '#f39c12', 'Low Fit': '#e74c3c'}
pie_colors = [color_map.get(l, 'gray') for l in label_counts.index]
wedges, texts, autotexts = axes[1].pie(
    label_counts.values, labels=label_counts.index,
    autopct='%1.1f%%', colors=pie_colors, startangle=90,
    textprops={'fontsize': 11})
for at in autotexts:
    at.set_fontweight('bold')
axes[1].set_title('Candidate Fit Category Breakdown', fontsize=13, fontweight='bold')

# Chart 3: Top 10 candidates ranked bar chart
top10 = ranked.head(10)
bar_colors_top = [color_map.get(l, 'gray') for l in top10['hybrid_fit_label']]
axes[2].barh(top10['candidate_name'][:-1], top10['hybrid_fit_score'][:-1],
             color=bar_colors_top[:-1], edgecolor='white')
axes[2].set_title('Top 10 Candidates by Fit Score', fontsize=13, fontweight='bold')
axes[2].set_xlabel('Final Hybrid Fit Score (%)')
axes[2].axvline(65, color='green', linestyle='--', lw=2, alpha=0.7)
for i, val in enumerate(top10['hybrid_fit_score'][:-1]):
    axes[2].text(val + 0.3, i, f'{val:.1f}', va='center', fontsize=9)

plt.suptitle(f'Resume Screening Results — {TARGET_ROLE} | TalentBridge Solutions',
            fontsize=15, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('final_results_dashboard.png', dpi=150, bbox_inches='tight')
plt.show()

```

Resume Screening Results — Data Scientist | TalentBridge Solutions



— 8.4 Business Recommendations for TalentBridge —

```

high_fit = df[df['hybrid_fit_label'] == 'High Fit']
medium_fit = df[df['hybrid_fit_label'] == 'Medium Fit']
low_fit = df[df['hybrid_fit_label'] == 'Low Fit']

```

```

after_nlp = len(high_fit) + len(medium_fit)
reduction = (1 - after_nlp / len(df)) * 100

print("=" * 70)
print("  BUSINESS RECOMMENDATIONS REPORT")
print(f"  Company : TalentBridge Solutions Pvt. Ltd.")
print(f"  Role    : {TARGET_ROLE}")
print(f"  Engine   : NLP Hybrid Scoring (Rule-Based + TF-IDF + ML)")
print("=" * 70)
print()
print("CANDIDATE POOL SUMMARY:")
print(f"  Total Screened      : {len(df):,} candidates")
print(f"  High Fit (>=65%)    : {len(high_fit):>4} ({len(high_fit)/len(df)*100:.1f}%) --> SHORTLIST IMMEDIATELY")
print(f"  Medium Fit (40-64%) : {len(medium_fit):>4} ({len(medium_fit)/len(df)*100:.1f}%) --> HUMAN REVIEW RECOMMENDED")
print(f"  Low Fit (<40%)      : {len(low_fit):>4} ({len(low_fit)/len(df)*100:.1f}%) --> DEPRIORITIZE / REJECT")
print()
print("EFFICIENCY GAINS:")
print(f"  Before NLP system   : {len(df):,} manual reviews needed")
print(f"  After NLP system    : {after_nlp:,} targeted reviews needed")
print(f"  Recruiter Workload Reduction: {reduction:.1f}%")
print(f"  Time Saved          : ~{reduction:.0f}% less manual screening time")
print()
print("SHORTLISTING THRESHOLDS (Recommended):")
print("  Score >= 65% : Auto-shortlist → Direct to hiring manager")
print("  Score 40-64% : Secondary review → Recruiter evaluates")
print("  Score < 40%  : Auto-reject or archive")
print()
print("KEY DRIVERS OF HIGH FIT SCORE:")
print(f"  1. Mandatory skill match (Python, Machine Learning, SQL, Statistics)")
print(f"  2. Relevant domain background (Data Science)")
print(f"  3. Experience >= {jd['min_experience']} years")
print(f"  4. High NLP contextual similarity to job description")
print(f"  5. Strong ATS flags (project management, agile, client-facing)")
print()
print("OPERATIONAL RECOMMENDATIONS:")
print("  - Integrate this engine into the existing ATS platform")
print("  - Retrain model quarterly as new resume patterns emerge")
print("  - Use explanation column for audit trail and compliance")
print("  - Monitor bias: ensure consistent scoring across geographies")
print()

```

```

=====
BUSINESS RECOMMENDATIONS REPORT
Company : TalentBridge Solutions Pvt. Ltd.
Role    : Data Scientist
Engine   : NLP Hybrid Scoring (Rule-Based + TF-IDF + ML)
=====

CANDIDATE POOL SUMMARY:
  Total Screened      : 5,000 candidates
  High Fit (>=65%)    : 238 (4.8%) --> SHORTLIST IMMEDIATELY
  Medium Fit (40-64%) : 758 (15.2%) --> HUMAN REVIEW RECOMMENDED
  Low Fit (<40%)      : 4004 (80.1%) --> DEPRIORITIZE / REJECT

EFFICIENCY GAINS:
  Before NLP system   : 5,000 manual reviews needed
  After NLP system    : 996 targeted reviews needed
  Recruiter Workload Reduction: 80.1%
  Time Saved          : ~80% less manual screening time

SHORTLISTING THRESHOLDS (Recommended):
  Score >= 65% : Auto-shortlist → Direct to hiring manager
  Score 40-64% : Secondary review → Recruiter evaluates
  Score < 40%  : Auto-reject or archive

KEY DRIVERS OF HIGH FIT SCORE:
  1. Mandatory skill match (Python, Machine Learning, SQL, Statistics)
  2. Relevant domain background (Data Science)
  3. Experience >= 3 years
  4. High NLP contextual similarity to job description
  5. Strong ATS flags (project management, agile, client-facing)

OPERATIONAL RECOMMENDATIONS:
  - Integrate this engine into the existing ATS platform
  - Retrain model quarterly as new resume patterns emerge
  - Use explanation column for audit trail and compliance
  - Monitor bias: ensure consistent scoring across geographies

BEST ML MODEL : Logistic Regression | Accuracy: 99.00%

```

Section 9: Conclusion

What We Built

A complete end-to-end NLP-powered Resume Screening and Role Matching system for TalentBridge Solutions that: